

date related functions

=====

```
SELECT current_date();
```

```
SELECT current_time();
```

```
SELECT current_timestamp();
```

```
SELECT DATE('2024-07-26 04:32:05') as date_only;
```

```
SELECT TIME('2024-07-26 04:32:05') as time_only;
```

```
SELECT EXTRACT(YEAR FROM '2024-07-26') as year_only;
```

```
SELECT EXTRACT(MONTH FROM '2024-07-26') as month_only;
```

```
SELECT EXTRACT(DAY FROM '2024-07-26') as day_only;
```

select

```
CONCAT(EXTRACT(YEAR FROM sale_date), '-', EXTRACT(MONTH FROM sale_date)) as yearmonth,
```

```
sum(amount) as total_sales
```

```
from sales
```

```
group by yearmonth;
```

select

```
EXTRACT(YEAR_MONTH FROM sale_date) AS yearmonth,
```

```
sum(amount) as total_sales
```

```
from sales
```

```
group by yearmonth;
```

YEAR/MONTH/DAY

```
SELECT MONTHNAME('2024-07-26 09:34:21');
```

-- DATE_ADD / DATE_SUB

```
select date_add(current_date, interval 10 day) as newdate;
```

```
select date_sub(current_date, interval 43 day) as newdate;
```

-- datediff

```
select datediff(current_date, '2024-06-13');
```

```
select datediff(current_date, '1992-02-13');
```

-- date_format

```
select date_format('2017-06-15', '%Y');
```

```
select date_format('2017-06-15', '%y');
```

```
select date_format('2017-06-15', '%m');
```

```
select date_format('2017-06-15', '%M');
```

```
select date_format('2017-06-15','%D');
```

```
select date_format('2017-06-15','%M %D %Y');
```

UNIX TIME - EPOCH TIME

NUMBER OF SECONDS SINCE 1ST JAN 1970

```
SELECT unix_timestamp();
```

```
SELECT from_unixtime(1721950195);
```

```
SELECT UNIX_TIMESTAMP('2024-06-13');
```

Numeric functions

other functions

```
SELECT ABS(-10);
```

```
SELECT CEIL(5.1);
```

```
SELECT FLOOR(5.9);
```

```
SELECT ROUND(4.567,2);
```

```
SELECT TRUNCATE(4.567,2);
```

```
SELECT MOD(10,3);
```

```
SELECT POWER(2,3);
```

```
SELECT SQRT(16);
```

```
SELECT SIGN(-5);
```

```
SELECT RAND();
```

other functions

=====

-- case

< 45000 then low

45000 and 55000 then medium

else high

```
select *,
```

```
CASE
```

```
    WHEN salary < 45000 THEN 'low'
```

```
    WHEN salary BETWEEN 45000 AND 55000 THEN 'medium'
```

```
    ELSE 'high'
```

```
END AS salary_band
```

```
    from employees;
```

```

select *,
CASE
    WHEN years_of_exp < 3 THEN 'low experience'
    WHEN years_of_exp < 8 THEN 'medium experience'
    WHEN years_of_exp < 15 THEN 'high experience'
    ELSE 'cant say'
END as experience_band
from students;

```

-- cast

casting from one datatype to another

'123'

```

CREATE TABLE `employees101` (
    `employee_id` int NOT NULL AUTO_INCREMENT,
    `first_name` varchar(50) NOT NULL,
    `last_name` varchar(50) NOT NULL,
    `department_id` int DEFAULT NULL,
    `salary` varchar(50) DEFAULT NULL,
    `hire_date` varchar(50) DEFAULT NULL,
    `manager_id` int DEFAULT NULL,
    `email` varchar(100) DEFAULT NULL,
    `phone_number` varchar(20) DEFAULT NULL,
    `job_title` varchar(50) DEFAULT NULL,
    PRIMARY KEY (`employee_id`)
)

```

```

insert into employees101
select * from employees;

```

```

select year(cast(hire_date as date))
from employees101;

```

```

select cast(salary as decimal(10,2)) * 1.1
from employees101;

```

-- coalesce

its a very useful function when you need to handle null values.

ensures that you have a default or fallback value.

```

select product_name,
coalesce(discount_price,regular_price) as display_price
from products;

```

```
select order_id,  
coalesce(billing_address,shipping_address,customer_address) as preferred_address  
from orders;
```

```
select sale_id,  
online_sales + store_sales AS total_sales  
from sales_new;
```

```
select sale_id,  
COALESCE (online_sales,0) + COALESCE(store_sales,0) AS total_sales  
from sales_new;
```

SCRIPTS

```
CREATE TABLE orders1 (  
    order_id INT AUTO_INCREMENT PRIMARY KEY,  
    customer_id INT,  
    product_id INT,  
    quantity INT,  
    order_date DATE,  
    total_amount DECIMAL(10, 2)  
);
```

```
INSERT INTO orders1 (customer_id, product_id, quantity, order_date, total_amount)  
VALUES  
    (1, 101, 2, CURRENT_DATE, 199.98),  
    (2, 102, 1, CURRENT_DATE, 99.99),  
    (3, 103, 5, CURRENT_DATE, 499.95),  
    (4, 104, 3, CURRENT_DATE, 299.97),  
    (5, 105, 4, CURRENT_DATE, 399.96);
```

```
CREATE TABLE products (  
    product_name VARCHAR(50),  
    discount_price DECIMAL(10, 2),  
    regular_price DECIMAL(10, 2)  
);
```

```
INSERT INTO products (product_name, discount_price, regular_price) VALUES  
('Laptop', NULL, 1200.00),  
('Smartphone', 800.00, 1000.00),  
('Tablet', NULL, 300.00);
```

```
CREATE TABLE orders (  
    order_id INT,  
    billing_address VARCHAR(100),  
    shipping_address VARCHAR(100),  
    customer_address VARCHAR(100)  
);
```

```
INSERT INTO orders (order_id, billing_address, shipping_address, customer_address) VALUES  
(1, '123 Main St', NULL, '789 Oak Ave'),  
(2, NULL, '456 Maple Rd', '101 Pine St'),  
(3, NULL, NULL, '303 Elm St');
```

```
CREATE TABLE sales_new (  
    sale_id INT,  
    online_sales DECIMAL(10, 2),  
    store_sales DECIMAL(10, 2)  
);
```

```
INSERT INTO sales_new (sale_id, online_sales, store_sales) VALUES  
(1, 1500.50, NULL),  
(2, NULL, 2500.75),  
(3, 1000.25, 500.00);
```